



Building a GPT *From Scratch*

A character-level transformer implemented from first principles in PyTorch. I grew it one component at a time to understand how language models actually work, then trained it on two corpora on a cloud GPU.

DISCIPLINE

Deep learning ·
Transformers · PyTorch
from first principles

STACK

PyTorch · NVIDIA A10G ·
Cloud training jobs

FORMAT

Component-by-
component walkthrough
of architecture, training
and results

What's Inside

01 **Executive Summary**
What it is, and the honest framing up front.

02 **Why Build It From Scratch**
Opening the black box the API hides.

03 **The Architecture, One Component at a Time**
Tokenizer to bigram to attention to the full block.

04 **Training**
The loop, and the scaled configuration.

05 **Running It on a GPU**
Why I ran training as a job, not a notebook.

06 **Results**
Shakespeare, invented dishes, and textbook overfitting.

07 **What the Model Is, and Is Not**
A next-character predictor, and nothing more.

08 **Honest Framing**
Why the understanding is the actual deliverable.

Executive Summary

I had used language models through APIs and fine-tuned an open model with a LoRA adapter, but I had never built the transformer or the training loop myself. The internals were a black box. This project closes that gap.

It is a small GPT implemented from first principles in PyTorch, built up one component at a time. I started with the tokenizer, then the attention mechanism, then the transformer block, then the training loop, then scaled it up and trained it on a GPU.

I trained it twice: on a public Shakespeare corpus, and on a corpus of about 17,000 public restaurant menu items I had scraped for a separate project. Trained on Shakespeare it produces Shakespeare-shaped text; trained on menus it invents plausible-looking dishes with ingredient lists and prices.

THE HONEST FRAMING UP FRONT

The goal was understanding, not a product. The model is a character-level next-token predictor. It does not understand meaning, follow instructions, or reason, and it is not production-grade. Its value is that I built every piece by hand, so none of it is a mystery anymore. That is the right foundation for the realistic way to ship a domain model: fine-tuning or distilling an open base model, not training one from scratch.

Why Build It From Scratch

Most applied AI work, correctly, treats the model as a given. You call an API, or at most you fine-tune. That is the right engineering choice for products, but it leaves the core a black box.

I wanted concrete answers to basic questions. What does attention actually compute, why does a transformer have the shape it does, and what does a training loop really do step by step. The only way to truly know is to build it. So I did, deliberately, from the bottom up, before letting myself use any reference implementation.

The Architecture, One Component at a Time

I followed the standard "build a GPT" progression and made sure each piece worked and made sense before adding the next.

STEP 1 · DATA

Tokenizer

Character-level: every unique character is assigned an integer. The objective is next-token prediction, so a chunk of text and the same chunk shifted right by one yield many (context, next-character) pairs at once. Split into train and validation so learning is told apart from memorizing.

STEP 2 · BASELINE

Bigram lookup

The simplest possible model: a lookup table mapping each character directly to next-character scores. A poor model, but it established the full pipeline (model, loss, training loop, generation) on something trivial before adding attention.

STEP 3 · COMMUNICATION

Self-attention

Each position is a weighted average of the positions before it. Each token emits a query and a key; their dot product sets the weights. A causal mask blocks seeing the future, scores are scaled before softmax, and a learned positional embedding restores order.

STEP 4 · COMPUTATION

Transformer block

Multi-head attention in parallel, then a per-position feed-forward network. Each is wrapped in a residual connection and preceded by layer norm. Together those are what make a deep stack trainable. The final model stacks several blocks.

THE MENTAL MODEL THAT MADE IT CLICK

Attention is communication: a token gathers information from other tokens. The feed-forward layer is computation: each token then thinks on what it gathered. Multi-head means several attention heads run in parallel, each free to specialize in a different relationship.

Training

The training loop is small and is the same engine regardless of model size: sample a batch, run a forward pass to a cross-entropy loss (a measure of how surprised the model is by the true next character), backpropagate to get the gradient, take an AdamW step, repeat. I tracked training and validation loss separately so I could see overfitting when it happened.

— The scaled configuration

11M	384	6 + 6	256
PARAMETERS	EMBEDDING DIM	HEADS + BLOCKS	CONTEXT WINDOW
Dropout	0.2		
Learning rate	3e-4		
Training steps	5,000		
Optimizer	AdamW		

The early experiments ran on a laptop CPU in minutes; the full model needed a GPU.

Running It on a GPU

I ran the full training as a managed cloud training job on an NVIDIA A10G GPU. It provisions the GPU, runs the script, writes the trained weights to storage, and shuts itself down. Each run took about twenty minutes and cost well under a dollar.

Results

On the Shakespeare corpus, validation loss fell from about 4.3 (random guessing over the character set) to about 1.50, and the output became coherent imitation:

SAMPLE · SHAKESPEARE CORPUS

```
Thy wring dave turn's mine,--  
As too, grost that is no owning in thy breast.  
Nay, three repair'd, Tybalt of foul laugh,  
First Senator:
```

On the menu corpus, the model learned the format and the culinary vocabulary and invented dishes that had never existed:

SAMPLE · MENU CORPUS

```
Enchiladas – Bean enchilada, whole cheese dip and guacamole ($16)  
Black & White Reuben – Porcho cheese sauce ($14)  
Green Chili de Con Poppino – Chipotle Sauce, scallions, white wine butter
```

AN HONEST OBSERVATION FROM THE MENU RUN

Training loss reached 0.68 while validation loss bottomed near 1.92 and then began to rise. That is textbook overfitting, caused by a small and repetitive corpus relative to an 11M-parameter model. The correct responses are early stopping at the best validation checkpoint, more regularization, or more data. Recognizing that is itself part of what the project taught.

What the Model Is, and Is Not

It is a character-level autoregressive text generator. Its single operation is predicting the next character, run in a loop, so it produces text that imitates the surface form of whatever it was trained on. You can let it free-run or seed it with starting text and it will continue.

It does not understand meaning, follow instructions, answer questions, or perform any task. The dishes it produces are plausible-looking but invented, and it has no knowledge beyond the character statistics of its corpus. It is a learning artifact and a demonstration, not a production system.

Honest Framing

The interesting result of this project is not a loss number or a clever sample. It is that the transformer stopped being a black box. Implementing attention, the residual stream, and the training loop, and then training the model on real hardware, turned "I use language models" into "I understand what they are doing."

THE ACTUAL DELIVERABLE

That understanding is the foundation for the realistic path to a production domain model: fine-tuning or distilling an open base model for narrow tasks, rather than training one from scratch. Knowing exactly why that is the right call, and not just being told it, is the point.

END